

Iterator Design Pattern (Behavioral)

Traverse a collection without knowing how it is stored.

Intent

The Iterator pattern provides a standard way to step through the elements of a collection sequentially without exposing the collection's internal data structure. It decouples the traversal algorithm from the collection itself so that the same client code can walk a list, a tree, or any other aggregate by talking only to the iterator interface. This also lets multiple independent traversals run simultaneously on the same collection because each iterator carries its own position state.

Real-world analogy

Think of a music streaming app's queue. When you press **Next**, you do not reach into the app's internal database or linked list to find which song comes after the current one — you simply press a button that says "give me the next track." The internal data structure (sorted by timestamp, stored in a ring buffer, backed by a graph, whatever) is completely irrelevant to you. That **Next** button is the iterator. It remembers where you are in the queue and moves forward on demand without you needing to know anything about how the playlist is arranged under the hood. This codebase models exactly that scenario.

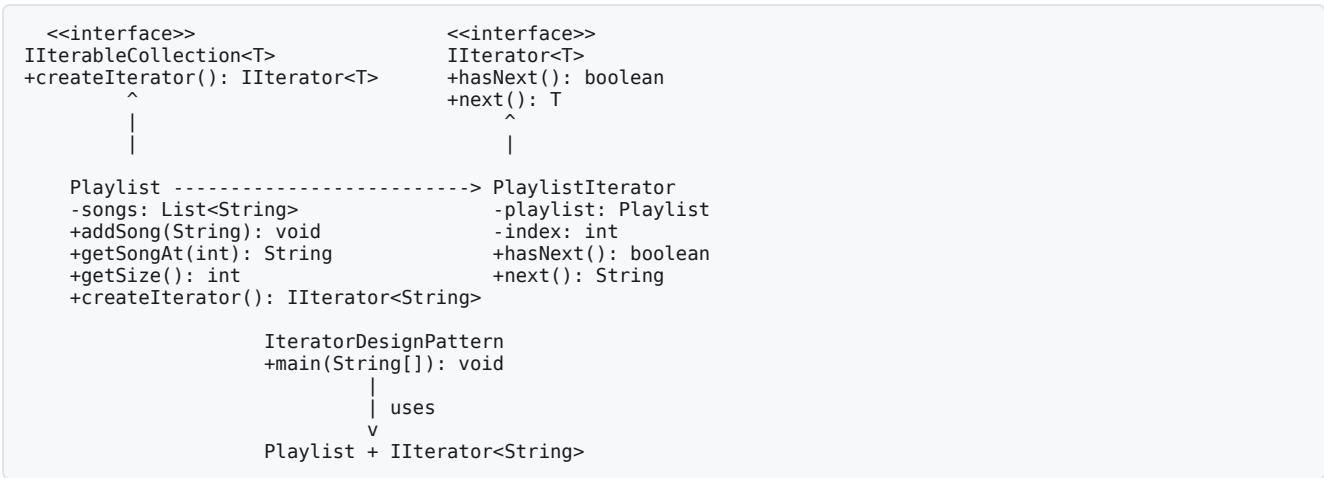
How the pattern works

Two parallel hierarchies collaborate:

Side	Role
Collection hierarchy	Knows how elements are stored; can produce an iterator that understands that storage.
Iterator hierarchy	Holds a cursor into the collection; exposes <code>hasNext()</code> and <code>next()</code> to advance.

The client talks only to the two interfaces — `IIterableCollection` and `IIterator` — and never touches the concrete `Playlist` or `PlaylistIterator` directly after the initial setup call. This is the Open/Closed principle: new collection types can be added without changing any iterator-consuming code.

UML class diagram (ASCII)



The players

- `IIterator<T>` — The *Iterator* interface. Declares the two-method contract (`hasNext` , `next`) that all iterators must honour. Typed with a generic `T` so it works for any element type.

- **IterableCollection<T>** — The *Aggregate* interface. Any collection that wants to be traversed implements this and returns an `IIterator<T>` from `createIterator()`.
- **Playlist** — The *ConcreteAggregate*. Holds the actual songs in an `ArrayList`, implements `IterableCollection<String>`, and manufactures a `PlaylistIterator` for itself.
- **PlaylistIterator** — The *ConcreteIterator*. Keeps a cursor (`index`) into the playlist and implements the traversal logic.
- **IteratorDesignPattern** — The *Client / entry point*. Builds a playlist, obtains an iterator, and consumes it without knowing anything about `ArrayList` or index arithmetic.

Code walkthrough — every line explained

IIterator.java

```
package com.design.patterns.iterator.contract;

public interface IIterator<T> {

    boolean hasNext();

    T next();

}
```

Line by line:

- `package com.design.patterns.iterator.contract;` — Declares that this compilation unit belongs to the `contract` sub-package, signalling that everything inside this package is a public API contract, not an implementation detail. Placing contracts in their own package makes it easy to depend on the abstraction layer in isolation.
- *(blank line)* — Visual separation between the package declaration and the type declaration; a Java style convention with no semantic effect.
- `public interface IIterator<T> {` — Declares a generic, publicly accessible interface named `IIterator`. The `<T>` type parameter is a placeholder for the element type the iterator will return; callers substitute a concrete type (e.g., `String`) at use-site, making this contract reusable for any collection element type. The opening brace `{` begins the interface body.
- *(blank line)* — Visual separation before the first method declaration.
- `boolean hasNext();` — Declares the *guard method* of the iterator contract. Any implementing class must provide a method that returns `true` when there are more elements remaining, `false` when the cursor has reached the end. Clients call this before every call to `next()` to avoid overrunning the collection.
- *(blank line)* — Visual separation between the two method declarations.
- `T next();` — Declares the *advance method*. Returns the element at the current cursor position and moves the cursor forward by one. The return type `T` is the same type parameter declared on the interface, so the caller receives a strongly typed element without casting.
- `}` — Closes the `IIterator<T>` interface body.

IIterableCollection.java

```
package com.design.patterns.iterator.contract;

public interface IIterableCollection<T> {

    IIterator<T> createIterator();

}
```

Line by line:

- `package com.design.patterns.iterator.contract;` — Places this interface in the same `contract` package as `IIterator`, keeping both halves of the abstract layer together and importable as a unit.
 - *(blank line)* — Visual separator before the type declaration.
 - `public interface IIterableCollection<T> {` — Declares the *Aggregate* interface. `public` makes it accessible from any package. The `<T>` type parameter flows through to the return type of `createIterator()`, ensuring that a collection of `String` produces an `IIterator<String>` — the types are bound at compile time with no unsafe casts. The `{` opens the interface body.
 - *(blank line)* — Visual separator before the method declaration.
 - `IIterator<T> createIterator();` — Declares the *factory method* of the aggregate contract. Any concrete collection that implements this interface must override this method and return an iterator wired to itself. By returning `IIterator<T>` rather than a concrete iterator class, the interface hides the iterator implementation from all callers — the client code only ever sees `IIterator<T>`.
 - `}` — Closes the `IIterableCollection<T>` interface body.
-

Playlist.java

```
package com.design.patterns.iterator.contract.concrete;

import java.util.ArrayList;
import java.util.List;

import com.design.patterns.iterator.contract.IIterator;
import com.design.patterns.iterator.contract.IIterableCollection;

public class Playlist implements IIterableCollection<String> {
    private final List<String> songs = new ArrayList<>();

    public void addSong(String song) {
        songs.add(song);
    }

    public String getSongAt(int index) {
        return songs.get(index);
    }

    public int getSize() {
        return songs.size();
    }

    @Override
    public IIterator<String> createIterator() {
        return new PlaylistIterator(this);
    }
}
```

Line by line:

- `package com.design.patterns.iterator.contract.concrete;` — Declares this file's package as `concrete` (a sub-package of `contract`), grouping all concrete implementations away from the interfaces while keeping them logically related under the same root.
- *(blank line)* — Separates the package declaration from the import block.
- `import java.util.ArrayList;` — Brings `ArrayList` (the JDK's resizable-array implementation of `List`) into scope so it can be referenced by simple name without the full qualified path. `ArrayList` is chosen here because songs are added sequentially and accessed by numeric index — operations that `ArrayList` handles in $O(1)$ amortised time.
- `import java.util.List;` — Brings the `List<E>` interface into scope. Declaring the field type as the interface (`List<String>`) rather than the concrete class (`ArrayList<String>`) means the field type is narrower than the implementation — a deliberate choice to hide the `ArrayList`-specific API from the rest of the class.
- *(blank line)* — Separates the JDK imports from the project imports.
- `import com.design.patterns.iterator.contract.IIterator;` — Brings the custom `IIterator<T>` interface into scope so `createIterator()` can reference it as its return type.
- `import com.design.patterns.iterator.contract.IIterableCollection;` — Brings `IIterableCollection<T>` into scope so `Playlist` can declare `implements IIterableCollection<String>`.
- *(blank line)* — Separates the import block from the class declaration.
- `public class Playlist implements IIterableCollection<String> {` — Declares the concrete aggregate. `public` makes it visible everywhere. `implements IIterableCollection<String>` binds the generic type parameter `T` to `String`, meaning this class promises to produce an `IIterator<String>`. The `{` opens the class body.
- `private final List<String> songs = new ArrayList<>();` — Declares the backing store. `private` hides the list from all external code — this is the information hiding that forces callers to use the iterator API instead of directly indexing the list. `final` prevents the field from being reassigned to a different `List` object after construction (the list's contents can still grow via `add`). `new ArrayList<>()` uses the diamond operator to infer the `String` type argument from the left-hand-side declaration.
- *(blank line)* — Visual separator before the first method.
- `public void addSong(String song) {` — Declares the *mutation method*. `public` so external code (the client / `main`) can populate the playlist. `void` because adding a song produces no return value. The parameter `song` is the song title string to be stored. The `{` opens the method body.
- `songs.add(song);` — Delegates to `ArrayList.add()` to append `song` at the end of the internal list. This is the only place where the outside world can change the list's contents, keeping mutation controlled.
- `}` — Closes the `addSong` method body.
- *(blank line)* — Visual separator between methods.
- `public String getSongAt(int index) {` — Declares the *indexed-read accessor*. `public` so `PlaylistIterator` (a different class, even though it is in the same package) can retrieve elements by position. Returning `String` directly rather than an `Object` is safe because the list is `List<String>`. The `{` opens the method body.
- `return songs.get(index);` — Delegates to `ArrayList.get(index)`, which returns the element at position `index` in $O(1)$ time. The iterator calls this method repeatedly, passing its cursor value.
- `}` — Closes the `getSongAt` method body.
- *(blank line)* — Visual separator between methods.
- `public int getSize() {` — Declares the *size accessor*. `public` so `PlaylistIterator.hasNext()` can compare the current cursor against the playlist length without accessing the private `songs` field directly. The `{` opens the method body.
- `return songs.size();` — Delegates to `ArrayList.size()`, returning the number of songs currently in the list as an `int`.

- `}` — Closes the `getSize` method body.
- *(blank line)* — Visual separator before the `createIterator` override.
- `@Override` — An annotation that instructs the compiler to verify that the method immediately below it actually overrides a method declared in a supertype. If the signature drifts (e.g., a typo in the method name), the compiler emits an error rather than silently creating an unrelated overload.
- `public Iterator<String> createIterator() {` — Implements the factory method required by `IterableCollection<String>`. Returns `Iterator<String>` (the interface type), not `PlaylistIterator` (the concrete type), so callers remain decoupled from the iterator implementation. The `{` opens the method body.
- `return new PlaylistIterator(this);` — Constructs a fresh `PlaylistIterator`, passing `this` (the current `Playlist` instance) as a back-reference so the iterator can call `getSongAt` and `getSize`. A new instance is created each time so that multiple iterators can operate independently on the same playlist.
- `}` — Closes the `createIterator` method body.
- `}` — Closes the `Playlist` class body.

PlaylistIterator.java

```
package com.design.patterns.iterator.contract.concrete;

import com.design.patterns.iterator.contract.IIterator;

public class PlaylistIterator implements IIterator<String> {
    private final Playlist playlist;
    private int index = 0;

    public PlaylistIterator(Playlist playlist) {
        this.playlist = playlist;
    }

    @Override
    public boolean hasNext() {
        return index < playlist.getSize();
    }

    @Override
    public String next() {
        return playlist.getSongAt(index++);
    }
}
```

Line by line:

- `package com.design.patterns.iterator.contract.concrete;` — Places `PlaylistIterator` in the same concrete package as `Playlist`, allowing it to call the package-visible aspects of `Playlist` and keeping both concrete classes as a cohesive unit.
- *(blank line)* — Separates the package declaration from the import block.
- `import com.design.patterns.iterator.contract.IIterator;` — Brings `IIterator<T>` into scope so `PlaylistIterator` can declare `implements IIterator<String>`.
- *(blank line)* — Separates the import block from the class declaration.
- `public class PlaylistIterator implements IIterator<String> {` — Declares the concrete iterator. `implements IIterator<String>` binds `T` to `String`, consistent with the `Playlist` it will traverse. The `{` opens the class body.
- `private final Playlist playlist;` — Holds a reference to the `Playlist` this iterator was created for. `private` encapsulates the back-reference; `final` ensures the iterator cannot be re-pointed at a different playlist after construction, which would break traversal invariants.
- `private int index = 0;` — The *cursor*: an integer that tracks the position of the next element to be returned. Initialised to `0` so traversal begins at the first song. `private` prevents external code from manipulating the cursor and corrupting the traversal.
- *(blank line)* — Visual separator before the constructor.
- `public PlaylistIterator(Playlist playlist) {` — The constructor. `public` so `Playlist.createIterator()` (in the same package) can call it. The parameter `playlist` is the collection this iterator will traverse. The `{` opens the constructor body.
- `this.playlist = playlist;` — Assigns the constructor parameter to the instance field. The `this.` prefix disambiguates the field from the parameter, both of which are named `playlist`.
- `}` — Closes the constructor body.
- *(blank line)* — Visual separator before the first interface method.
- `@Override` — Instructs the compiler to verify that `hasNext()` overrides a method in `IIterator<String>`. Protects against typos in the method name.
- `public boolean hasNext() {` — Implements the guard method from `IIterator`. `public` is required because the interface method is `public`. Returns `boolean`. The `{` opens the method body.
- `return index < playlist.getSize();` — The core guard predicate. `index` is the position of the *next* element to return. If `index` is strictly less than the total number of songs, at least one more element exists and the method returns `true`. When `index` equals `getSize()` (i.e., the cursor has moved past the last element), it returns `false`, stopping the while-loop in the client.
- `}` — Closes the `hasNext` method body.
- *(blank line)* — Visual separator between the two iterator methods.
- `@Override` — Same compiler-check annotation for the `next()` override.

- `public String next() {` — Implements the advance method from `Iterator`. Returns `String` because `T` is bound to `String`. The `{` opens the method body.
- `return playlist.getSongAt(index++);` — The traversal step. `index++` is *post-increment*: the current value of `index` is passed to `getSongAt()` first (so the correct element is retrieved), and then `index` is incremented by 1 to prepare the cursor for the next call. This is the idiomatic way to return-then-advance in a single expression, avoiding a separate `index = index + 1` statement.
- `}` — Closes the `next` method body.
- `}` — Closes the `PlaylistIterator` class body.

IteratorDesignPattern.java

```
package com.design.patterns.iterator;

import com.design.patterns.iterator.contract.IIterator;
import com.design.patterns.iterator.contract.concrete.Playlist;

public class IteratorDesignPattern {

    public static void main(String[] args) {
        System.out.println("Iterator Design Pattern");

        Playlist playlist = new Playlist();
        playlist.addSong("Shape of You");
        playlist.addSong("Bohemian Rhapsody");
        playlist.addSong("Blinding Lights");

        IIterator<String> iterator = playlist.createIterator();

        System.out.println("Now Playing:");
        while (iterator.hasNext()) {
            System.out.println(" 🎵 " + iterator.next());
        }
    }
}
```

Line by line:

- `package com.design.patterns.iterator;` — Places the entry-point class in the root `iterator` package, one level above the `contract` and `concrete` sub-packages. This mirrors a typical project structure where the launcher lives at the root and imports from the sub-packages.
- *(blank line)* — Separates the package declaration from the import block.
- `import com.design.patterns.iterator.contract.IIterator;` — Brings `IIterator<T>` into scope. The variable `iterator` is typed as this interface, not as `PlaylistIterator`, demonstrating that the client depends only on the abstraction.
- `import com.design.patterns.iterator.contract.concrete.Playlist;` — Brings `Playlist` into scope so it can be instantiated. `PlaylistIterator` is intentionally *not* imported here — the client never references the concrete iterator class by name; it receives an `IIterator<String>` through the factory method.
- *(blank line)* — Separates the import block from the class declaration.
- `public class IteratorDesignPattern {` — Declares the public entry-point class. The class name matches the file name (a Java requirement). The `{` opens the class body.
- *(blank line)* — Visual separator before the `main` method.
- `public static void main(String[] args) {` — The JVM entry point. `public` so the JVM can call it from outside the class. `static` so it can be invoked without instantiating `IteratorDesignPattern`. `void` because the method has no return value to the JVM. `String[] args` captures any command-line arguments (none are used here). The `{` opens the method body.
- `System.out.println("Iterator Design Pattern");` — Prints the banner line to standard output followed by a newline. This is the first line that appears in the console when the program runs, identifying which pattern is being demonstrated.
- *(blank line)* — Visual separator in the source before the playlist setup block.
- `Playlist playlist = new Playlist();` — Creates a new, empty `Playlist` instance. The variable is typed as `Playlist` (the concrete class) because the client *needs* to call `addSong()`, which is not part of the `IterableCollection` interface. After population, the client will switch to the abstract `IIterator` type.
- `playlist.addSong("Shape of You");` — Calls `Playlist.addSong`, which appends the string "Shape of You" to the internal `ArrayList`. This becomes element at index 0.
- `playlist.addSong("Bohemian Rhapsody");` — Appends "Bohemian Rhapsody" to the list. This becomes element at index 1.
- `playlist.addSong("Blinding Lights");` — Appends "Blinding Lights" to the list. This becomes element at index 2. The list now has size 3.
- *(blank line)* — Visual separator before the iterator creation.
- `IIterator<String> iterator = playlist.createIterator();` — Calls the factory method on the playlist. `playlist.createIterator()` returns `new PlaylistIterator(this)` — but here the variable is declared as `IIterator<String>`, so the concrete type `PlaylistIterator` is invisible to all code below this line. This is the key decoupling moment: from here on, the loop works against the interface alone.
- *(blank line)* — Visual separator before the traversal block.
- `System.out.println("Now Playing:");` — Prints the section header to standard output followed by a newline. Appears immediately before the song listing.

- `while (iterator.hasNext()) {` — Begins the traversal loop. The condition calls `IIterator.hasNext()` on each iteration. The loop body executes as long as the iterator reports more elements available. The `{` opens the loop body.
- `System.out.println(" 🎵 " + iterator.next());` — The single statement in the loop body. `iterator.next()` retrieves the song at the current cursor position and advances the cursor. The string `" 🎵 "` (a space, a musical note emoji, and a space) is prepended to the song title using string concatenation before printing to standard output. `println` appends a newline after each song.
- `}` — Closes the `while` loop body. Control returns to the condition check (`iterator.hasNext()`).
- `}` — Closes the `main` method body.
- `}` — Closes the `IteratorDesignPattern` class body.

Why these design decisions

Why the Iterator pattern here

A playlist is inherently a sequence — clients need to walk it from start to finish. But clients should not be coupled to the fact that `Playlist` uses an `ArrayList` internally. If the storage ever changes (a linked list, a database cursor, a remote stream), the client code stays unchanged as long as the iterator contract is honoured. The pattern also opens the door to multiple iterator flavours later (reverse iterator, shuffle iterator, filtered iterator) without modifying `Playlist` or the client at all.

Why two separate interfaces (`IIterator` and `IIterableCollection`)

Splitting the contracts separates two distinct concerns: *how to traverse* (`IIterator`) and *how to produce a traversal* (`IIterableCollection`). This allows, for example, a class to implement `IIterableCollection` but delegate traversal to a completely different class hierarchy, or for an iterator to outlive the scope where the collection was created.

Why `Playlist` exposes `getSongAt` and `getSize` rather than giving the iterator direct field access

`PlaylistIterator` is in a separate class and therefore cannot access `Playlist`'s private field `songs` directly. Providing narrow accessor methods (`getSongAt` , `getSize`) instead of making the field package-private or public keeps the list fully encapsulated. Only the two operations the iterator actually needs are exposed — nothing more.

Why `index++` (post-increment) inside `getSongAt(index++)`

Post-increment evaluates to the *current* value for the expression and then increments. This idiom retrieves the element at the current position *and* moves the cursor forward in one line, exactly as an iterator must do. The alternative — a separate `index = index + 1` below the return — would not compile (code after `return` is unreachable in Java).

Why `this` is passed to the `PlaylistIterator` constructor

The iterator needs to query the playlist for element values and size, but should not own the data. Passing `this` gives the iterator a reference to call the playlist's accessor methods, keeping the data in one place (the playlist) while the cursor state lives in the iterator. This also means two iterators created from the same playlist share the data without duplication.

Trade-offs

Benefit	Cost
Client is fully decoupled from storage structure	Extra classes per collection type
Multiple concurrent iterators on the same collection	Iterator becomes invalid if the collection is structurally modified mid-traversal (no fail-fast here unlike <code>java.util.Iterator</code>)
New iterator strategies (reverse, filtered) are open for addition	Iterator must trust that <code>getSongAt</code> is safe for the index range it calculated
Strongly typed — no casting at call sites	Requires Java generics literacy to read

Execution flow (step-by-step trace of `main()`)

1. JVM calls `main(String[])`.
2. `System.out.println("Iterator Design Pattern")`
-> prints "Iterator Design Pattern\n"
3. `new Playlist()`
-> constructs `Playlist`; `songs = []` (empty `ArrayList`)

```

4. playlist.addSong("Shape of You")
   -> songs = ["Shape of You"]

5. playlist.addSong("Bohemian Rhapsody")
   -> songs = ["Shape of You", "Bohemian Rhapsody"]

6. playlist.addSong("Blinding Lights")
   -> songs = ["Shape of You", "Bohemian Rhapsody", "Blinding Lights"]

7. playlist.createIterator()
   -> new PlaylistIterator(playlist)
       PlaylistIterator.playlist = <Playlist ref>
       PlaylistIterator.index    = 0
   -> returned as Iterator<String>

8. System.out.println("Now Playing:")
   -> prints "Now Playing:\n"

--- Loop iteration 1 ---
9. iterator.hasNext() -> index(0) < getSize()(3) -> true -> enter loop
10. iterator.next()   -> getSongAt(0) -> "Shape of You"; index becomes 1
11. System.out.println(" 🎵 Shape of You")

--- Loop iteration 2 ---
12. iterator.hasNext() -> index(1) < 3 -> true
13. iterator.next()    -> getSongAt(1) -> "Bohemian Rhapsody"; index becomes 2
14. System.out.println(" 🎵 Bohemian Rhapsody")

--- Loop iteration 3 ---
15. iterator.hasNext() -> index(2) < 3 -> true
16. iterator.next()    -> getSongAt(2) -> "Blinding Lights"; index becomes 3
17. System.out.println(" 🎵 Blinding Lights")

--- Loop check 4 ---
18. iterator.hasNext() -> index(3) < 3 -> false -> exit loop

19. main() returns; JVM exits.

```

Expected output

```

Iterator Design Pattern
Now Playing:
🎵 Shape of You
🎵 Bohemian Rhapsody
🎵 Blinding Lights

```

How to run

```

# From the Iterator_Design_Pattern module root
mvn clean package

java -cp target/classes com.design.patterns.iterator.IteratorDesignPattern

```

If building from the parent aggregator:

```

# From Design-Pattern-G04 root
mvn clean package -pl Behavioral/Iterator_Design_Pattern

java -cp Behavioral/Iterator_Design_Pattern/target/classes \
    com.design.patterns.iterator.IteratorDesignPattern

```

Other real-world problems this pattern can solve

Domain	Collection	What the iterator hides
File system walker	Directory tree	Recursive DFS vs BFS vs OS <code>readdir</code> calls
Database result set	Query results	JDBC cursor, pagination, lazy loading
Social-media feed	Post stream	Pagination tokens, API rate-limit retries, caching
GUI component tree	Widget hierarchy	Tree traversal order, visibility filtering

Domain	Collection	What the iterator hides
Build pipeline	Task graph	Topological sort, dependency resolution
Log processing	Log file	Line buffering, gzip decompression, remote fetch
Shopping cart checkout	Cart items	Discount rule application order, availability checks
Notification queue	Alert list	Priority ordering, deduplication, delivery-status filtering

In every case the client loop is identical — `while (it.hasNext()) { process(it.next()); }` — and the complexity lives inside the iterator, fully hidden from the consumer.